Contents

Day 24 — MVP Design

From Product Hypothesis to an Executable Specification

By Day 24, the objective changes again.

The previous days focused on identifying valuable problems and discovering what becomes possible when intelligence is cheap. Today the task is to turn one of those opportunities into a **concrete product specification that an engineering system can implement**.

This is the bridge between product thinking and AI engineering.

The central principle is:

> **A good MVP specification converts an ambiguous product idea into a constrained, testable system.**

The output is not a collection of feature ideas.

It is an engineering artifact containing:

User + Problem + Workflow + Requirements + Architecture + Metrics + Evaluation + Scope

The final specification should be sufficiently precise that a coding agent—or a human engineering team—can implement the first version without having to rediscover the product.

---

1. What an MVP Actually Is

MVP stands for **Minimum Viable Product**.

The word "minimum" is often misunderstood.

An MVP is not:

> "The smallest amount of code we can write."

It is:

> **The smallest complete system capable of testing the core product hypothesis with real users.**

This distinction is critical.

Suppose the hypothesis is:

> "Engineers will use an AI system that investigates production incidents and reduces investigation time."

An MVP might require:

- log ingestion,
- deployment history,
- incident context,
- AI analysis,
- evidence citations,
- a structured investigation report,
- human review,
- evaluation,
- basic observability.

It does **not** necessarily require:

- a sophisticated mobile application,
- ten integrations,
- autonomous remediation,
- enterprise billing,
- advanced customization,
- a fully generalized agent framework.

The correct optimization target is:

$$\boxed{\text{minimize Cost} \quad \text{subject to} \quad \text{Hypothesis Testability} \geq \tau}$$

where τ represents the minimum evidence needed to test the product hypothesis.

---

2. Choose One Idea

The first step is to choose **one** opportunity from the previous exercise.

Do not attempt to build a platform.

Do not create a generic:

> "AI assistant for everything."

Select a narrow problem with:

- a specific user,
- a specific workflow,
- a measurable pain point,
- a plausible AI advantage,
- a clear outcome.

For example:

> **AI Production Incident Investigator**

Target user:

> Software engineers responsible for investigating production incidents.

Problem:

> Incident investigation requires manually correlating logs, deployments, tickets, metrics, and recent code changes.

Desired outcome:

> Produce a reliable, evidence-backed incident analysis significantly faster than a human performing the investigation manually.

This is sufficiently constrained to become an engineering problem.

---

3. User Persona

The persona describes the person actually using the system.

Avoid demographic descriptions that do not affect product behavior.

For engineering purposes, a useful persona specifies:

Role

Who performs the workflow?

Goals

What are they trying to accomplish?

Environment

What systems and tools do they already use?

Constraints

What prevents them from solving the problem easily?

Expertise

What do they already understand?

Success criteria

What would make them consider the product useful?

For example:

Persona: Production Engineer

- Works on a software engineering team.
- Participates in on-call rotations.
- Has access to logs, metrics, deployment systems, and incident tickets.
- Frequently investigates production failures under time pressure.
- Needs evidence before accepting a proposed root cause.
- Values speed but cannot tolerate fabricated explanations.
- Wants the system to reduce investigation effort without removing human control.

The persona immediately informs architecture.

For example, because the user requires evidence, the system should not merely produce:

> "The database caused the outage."

It should produce:

> "The database connection pool exhausted at 14:32 UTC. Evidence: metrics X, log entries Y, and deployment Z."

The persona therefore drives product requirements.

---

4. Problem Statement

A strong problem statement should be specific enough to test.

A useful structure is:

> [**User**] experiences [**problem**] when [**situation**], resulting in [**cost/consequence**].

For example:

> Production engineers spend significant time correlating logs, metrics, deployment history, and incident tickets during production failures, resulting in slow incident resolution and substantial engineering overhead.

Then define the desired transformation:

$$\text{Current State} \rightarrow \text{Desired State}$$

Current:

Incident → Manual Investigation → Hypotheses → Evidence Gathering → Report

Desired:

Incident → AI Investigation → Evidence-backed Hypotheses → Human Verification → Report

The MVP exists to determine whether the second workflow is materially better.

---

5. Product Hypothesis

Before designing the system, write the hypothesis explicitly.

For example:

> If production engineers can provide an incident identifier and receive an evidence-backed analysis of relevant logs, metrics, deployments, and tickets within several minutes, then they will use the system during incident investigation and achieve a substantial reduction in investigation time.

This can be formalized as:

$$H = H_{\text{problem}} \wedge H_{\text{solution}} \wedge H_{\text{adoption}} \wedge H_{\text{economics}}$$

The MVP should produce evidence about these hypotheses.

This is important because an MVP is fundamentally an **experiment**.

---

6. Workflow

The workflow describes what happens from user intent to outcome.

A useful representation is:

$$W = (s_1, s_2, \dots, s_n)$$

For our example:

Step 1 — User identifies incident

The engineer enters an incident ID or describes the incident.

Step 2 — System gathers context

The system retrieves:

- incident metadata,
- recent deployments,
- relevant logs,
- metrics,
- tickets,
- recent code changes.

Step 3 — System forms hypotheses

The AI analyzes the evidence and identifies plausible causes.

Step 4 — System investigates

The agent performs additional searches or tool calls.

Step 5 — System verifies

The system checks whether proposed explanations are supported by evidence.

Step 6 — System generates report

The system produces:

- summary,
- timeline,
- suspected causes,
- supporting evidence,
- conflicting evidence,
- uncertainty,
- recommended next steps.

Step 7 — Human reviews

The engineer accepts, rejects, or modifies the conclusions.

This produces a complete workflow:

Intent → Context → Investigation → Verification → Output → Human Review

---

7. Product Requirements

The next step is to convert the workflow into explicit requirements.

Requirements should describe **observable system behavior**, not implementation preferences.

For example:

Functional requirements

The system must:

1. Accept an incident identifier.
2. Retrieve incident metadata.
3. Retrieve relevant logs.
4. Retrieve deployment information.
5. Retrieve relevant metrics.
6. Search incident-related tickets.
7. Construct an investigation context.
8. Generate candidate hypotheses.
9. Perform additional tool calls when necessary.
10. Produce an evidence-backed report.
11. Cite supporting evidence.
12. Express uncertainty explicitly.
13. Allow human review.
14. Preserve the investigation trace.

Non-functional requirements

The system should:

- respond within a defined latency target,
- provide deterministic structured outputs where appropriate,
- maintain audit logs,
- enforce tool permissions,
- protect sensitive data,
- expose failures clearly,
- support reproducible evaluation.

The distinction is important:

$$\text{Feature List} \neq \text{Product Requirements}$$

A feature list says:

> "Add RAG."

A requirement says:

> "The system must retrieve the most relevant incident evidence and make the evidence traceable to its source."

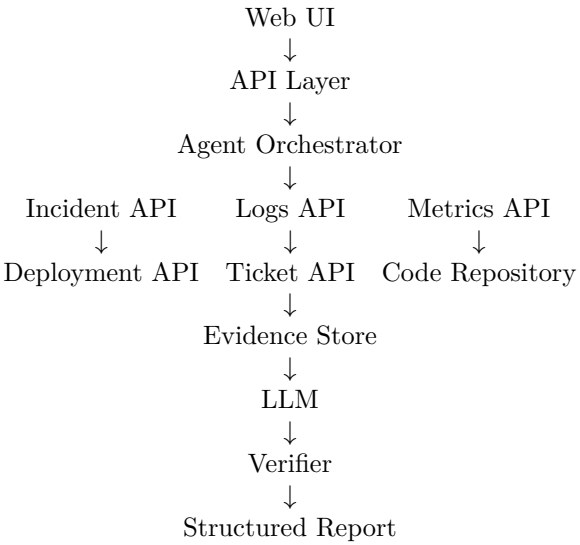
The latter describes a user-visible property.

---

8. Architecture

Only after the workflow and requirements are defined should the architecture be specified.

A reasonable MVP architecture might be:



The architecture should explicitly identify:

- state,
- context,
- tools,
- model calls,
- retrieval,
- verification,
- persistence,
- observability.

A useful generic architecture is:

User → Intent → Context Construction → Agent → Tools → Evidence → Verification → Structured Output

This connects directly to the concepts developed earlier in the course.

---

9. Context Engineering

The architecture should not simply send every available piece of information to the model.

Context must be constructed deliberately.

Let:

$$C = C_{\text{system}} + C_{\text{user}} + C_{\text{state}} + C_{\text{retrieved}} + C_{\text{tool}}$$

The context construction subsystem determines which information enters the model.

For incident investigation, this might include:

$$C = \text{incident}, \text{timeline}, \text{recent deployments}, \text{relevant logs}, \text{metrics}, \text{code changes}$$

The engineering problem is therefore not:

> "How do we put all the data into the prompt?"

It is:

> **"How do we construct the smallest sufficient context for reliable reasoning?"**

This is a central property of production AI systems.

---

10. Tool Design

Agentic products require explicit tool boundaries.

For example:

```
get_incident(id)
search_logs(query, time_range)
get_metrics(metric, time_range)
get_deployments(time_range)
search_tickets(query)
get_code_changes(time_range)
```

Each tool should have:

- a precise schema,
- explicit permissions,
- bounded scope,
- error handling,
- observability,
- predictable semantics.

The agent should not receive arbitrary access to the environment.

A useful security principle is:

$$\boxed{\text{Agent Capability} \subseteq \text{Explicitly Authorized Tools}}$$

The MVP should also distinguish between:

Read-only tools

Safe investigation operations.

Mutating tools

Actions that change external state.

For the first MVP, read-only tools are often sufficient.

This dramatically reduces risk.

---

11. Structured Outputs

AI-native systems should avoid relying exclusively on free-form text.

Define an explicit output schema.

For example:

```
IncidentReport
    summary
    timeline[]
    hypotheses[]
        hypothesis
        confidence
        supporting_evidence[]
        conflicting_evidence[]
    root_cause
    uncertainty[]
    recommendations[]
```

This gives downstream components a stable interface.

Formally:

$$LLM(x) \to y \in \mathcal{Y}$$

where $\mathcal{Y}$ is a constrained output space.

Structured outputs improve:

- validation,
- evaluation,
- UI rendering,
- storage,
- downstream automation,
- reproducibility.

---

12. Verification

For AI products, generation is only half of the system.

A robust architecture is:

$$\text{Generate} \to \text{Verify} \to \text{Accept/Reject}$$

For the incident investigator, verification might check:

Evidence grounding

Does each important claim have supporting evidence?

Temporal consistency

Does the proposed cause occur before the observed failure?

Source consistency

Do multiple sources support the explanation?

Contradiction detection

Does any evidence contradict the hypothesis?

Schema validity

Is the output structurally valid?

Confidence calibration

Does expressed confidence correspond reasonably to evidence strength?

This creates:

$$\text{Agent} + \text{Verifier}$$

rather than:

$$\text{Agent} \rightarrow \text{Trust Whatever It Says}$$

---

13. Success Metrics vs. Evaluation Metrics

These are different concepts and should not be conflated.

Success metrics

Measure whether the **product creates value**.

Examples:

- investigation time reduction,
- percentage of incidents successfully analyzed,
- user adoption,
- repeat usage,
- engineer satisfaction,
- percentage of reports accepted without major correction.

For example:

$$M_{\text{time}} = \frac{T_{\text{manual}} - T_{\text{AI}}}{T_{\text{manual}}}$$

If manual investigation takes 60 minutes and AI-assisted investigation takes 20 minutes:

$$M_{\text{time}} = \frac{60 - 20}{60} = 66.7\%$$

The product has demonstrated a potentially meaningful workflow improvement.

---

14. Evaluation Metrics

Evaluation metrics measure whether the **AI system performs correctly**.

Examples include:

- retrieval precision,
- retrieval recall,
- evidence coverage,
- factual accuracy,
- groundedness,
- hallucination rate,
- tool-call success rate,
- hypothesis accuracy,
- citation correctness,
- structured-output validity,
- latency,
- token consumption,
- cost per investigation.

A useful conceptual separation is:

$$\boxed{\text{Product Metrics} \neq \text{Model Metrics}}$$

A system can achieve excellent model metrics while providing little product value.

Conversely, a product can create substantial value despite imperfect model-level performance if the workflow is designed to contain errors.

---

15. Define the Evaluation Dataset Early

Do not wait until after implementation to construct evaluations.

Create a small **golden dataset** before building the system.

For example:

$$D = (d_1, y_1), (d_2, y_2), \ldots, (d_n, y_n)$$

where each d_i is an incident and y_i contains expected properties such as:

- important evidence,
- plausible root causes,
- temporal relationships,
- known irrelevant signals,

- expected uncertainty.

The evaluation harness can then run:

$$System(D) \rightarrow \hat{Y}$$

and compare:

$$\hat{Y} \quad \text{vs.} \quad Y$$

This creates a regression mechanism for the AI system.

---

16. MVP Scope

Scope is one of the most important engineering decisions.

The MVP should deliberately exclude functionality.

For our example:

In scope

- one incident-management source,
- one log source,
- one metrics source,
- deployment history,
- read-only investigation,
- one LLM provider,
- structured incident reports,
- evidence citations,
- basic web interface,
- evaluation harness,
- observability.

Out of scope

- autonomous remediation,
- multi-cloud support,
- ten different observability platforms,
- mobile application,
- enterprise SSO,
- advanced billing,
- generalized autonomous coding,
- fully autonomous incident resolution.

This distinction is essential.

A useful formula is:

$$MVP = \text{Minimum Complete Workflow}$$

not:

$$MVP = \text{Minimum Feature Count}$$

---

17. The MVP Boundary

A useful technique is to define the system boundary explicitly.

For example:

$$\boxed{\text{User} \rightarrow \underbrace{\text{AI Investigation System}}_{\text{MVP}} \rightarrow \text{Evidence-backed Report}}$$

Everything outside the boundary is initially treated as an external dependency.

This prevents architectural scope explosion.

You do not need to build:

- a new logging platform,
- a new ticketing system,
- a new metrics database,
- a new code repository.

You need to integrate with existing systems.

This is particularly important for AI products because the number of possible integrations is effectively unlimited.

---

18. What Should the Coding Agent Receive?

The final artifact of today's exercise is a **coding-agent specification**.

The coding agent should not receive:

> "Build an AI incident investigator."

That instruction leaves hundreds of architectural decisions unresolved.

Instead, give it a structured specification containing:

```
Product
    Purpose
    Target user
    Problem

Workflow
    Step 1
    Step 2
    Step 3
    ...

Requirements
    Functional
    Non-functional

Architecture
    Components
    Data flow
    Tools
    State
    Model
    Verification

Interfaces
    API schemas
    Tool schemas
    Output schemas

Evaluation
    Dataset
    Metrics
    Acceptance criteria

Success Criteria
    Product metrics
    User metrics

MVP Scope
    In scope
    Out of scope

Engineering Constraints
    Security
    Privacy
    Cost
    Latency
```

The specification becomes the interface between **product reasoning and implementation**.

---

19. Specification as a Contract

This is an important conceptual shift.

A product specification should function as a contract between:

Human Intent

and

Implementation System

The coding agent receives:

$$S = R, A, I, E, M, C$$

where:

- R = requirements,
- A = architecture,
- I = interfaces,
- E = evaluation,
- M = metrics,
- C = constraints.

The coding agent then produces:

$$Implementation = f(S)$$

The better the specification, the smaller the gap between intended and implemented behavior.

This becomes increasingly important as coding agents become capable of producing large quantities of software.

The bottleneck shifts from:

> "Can we write the code?"

toward:

> **"Can we specify exactly what should be built?"**

---

20. Coding Agents Change the Economics of Specification

When human engineers write every line of implementation, requirements can remain somewhat implicit because the engineer carries significant contextual knowledge.

With coding agents, this assumption becomes dangerous.

The agent does not necessarily know:

- which behavior matters most,
- which tradeoffs are acceptable,
- what should not be built,
- how success will be measured,
- what constitutes a safe failure,
- what the user actually values.

Therefore:

$$\text{More Capable Coding Agent} \Rightarrow \text{Greater Value of Precise Specifications}$$

The coding agent amplifies implementation capability.

It does not automatically amplify product judgment.

That remains a human responsibility.

---

21. Acceptance Criteria

Every major requirement should have an acceptance criterion.

For example:

Requirement

The system must provide evidence for important conclusions.

Acceptance criterion

For every root-cause hypothesis in the golden evaluation dataset, the system must identify at least one correct supporting evidence item or explicitly classify the hypothesis as unsupported.

This transforms:

Requirement

into:

Requirement + Test

A strong specification therefore creates a direct path:

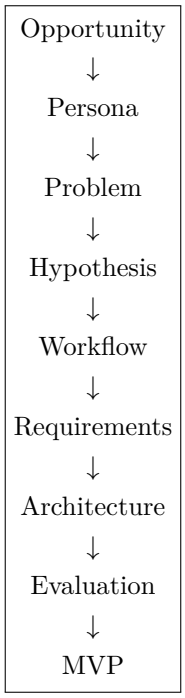
Requirement → Test → Implementation → Evaluation

This is particularly powerful for AI systems because many requirements can otherwise remain subjective.

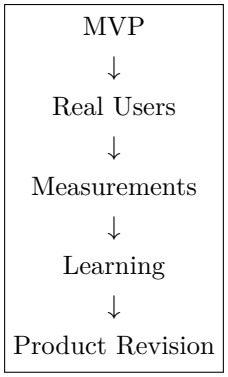
---

22. The Complete MVP Loop

The entire Day 24 process can be represented as:



Then:



The important point is that the MVP is not the end of the process.

It is the first instrument for learning.

---

23. Day 24 Exercise

Take **one** of the AI opportunities identified on Day 22 or Day 23.

Create a complete MVP specification containing the following sections.

1. User Persona

Define:

- role,
- goals,
- environment,
- constraints,
- expertise,
- success criteria.

2. Problem Statement

Describe:

- current workflow,
- pain point,
- frequency,
- economic cost,
- desired outcome.

3. Product Hypothesis

State what you believe the product will accomplish and what assumptions must be validated.

4. Workflow

Describe the complete workflow:

$$\text{User Intent} \rightarrow \text{System Processing} \rightarrow \text{Actions} \rightarrow \text{Output}$$

5. Product Requirements

Separate:

- functional requirements,
- non-functional requirements,
- security requirements,
- operational requirements.

6. Architecture

Specify:

- frontend,
- API,
- orchestration,
- models,
- retrieval,
- tools,
- state,
- storage,
- verification,
- observability.

7. Interfaces

Define:

- APIs,
- tool schemas,
- structured outputs,
- data models.

8. Success Metrics

Define product-level metrics such as:

Time Saved

Task Completion Rate

Adoption

User Satisfaction

9. Evaluation Metrics

Define AI/system-level metrics such as:

- accuracy,
- groundedness,
- retrieval quality,
- tool-call success,
- hallucination rate,
- latency,
- cost.

10. Evaluation Dataset

Create a small golden dataset before implementation.

11. MVP Scope

Explicitly define:

In scope

and

Out of scope.

12. Acceptance Criteria

For each major requirement, specify a test that determines whether it has been satisfied.

---

24. Final Deliverable: The Coding-Agent Specification

The final deliverable should be a document that can be handed directly to a coding agent.

It should answer:

What are we building?

Who is it for?

What problem does it solve?

How does the workflow operate?

What exactly must the system do?

What architecture should implement it?

How will we know whether it works?

What is explicitly excluded?

The specification should be precise enough that the coding agent can move directly from:

Specification

to:

Repository → Implementation → Tests → Evaluation

without inventing the product itself.

That is the central skill being developed.

Key Takeaways

1. **An MVP is the smallest complete system capable of testing a product hypothesis.** It is not simply the smallest amount of code.
2. **Product specification precedes implementation.** The engineering team or coding agent should not be responsible for discovering the product while writing it.
3. **Start with the user and workflow.** Architecture should emerge from the problem rather than determine the problem.

4. **Separate product metrics from AI evaluation metrics.** Time saved and adoption measure product value; groundedness, accuracy, retrieval quality, and tool success measure system behavior.
5. **Define evaluation before implementation.** A golden dataset provides a stable target against which the system can be measured.
6. **AI products require explicit verification.** Generation without validation is insufficient for high-value workflows.
7. **Scope is a first-class engineering decision.** A narrow, complete workflow is generally more valuable than a broad collection of unfinished features.
8. **Acceptance criteria convert product requirements into engineering tests.**
9. **Coding agents increase the importance of specification.** As implementation becomes cheaper, product judgment and precise system definition become more valuable.
10. **The specification is the bridge between product and engineering.**

Product Hypothesis → Specification → Coding Agent → Working System → Evaluation

11. **The MVP is an experiment, not a final product.** Its purpose is to generate evidence about whether the problem, solution, economics, and workflow are actually valid.
12. **The ultimate goal is to make implementation downstream of intent.** The product team defines the desired outcome, constraints, and evidence of success; the coding agent turns that specification into an executable system.